

TrustScan: AI & ML Architecture Report

Technical Implementation & Calculation Logic

1. Core Detection Algorithms

- **Logistic Regression Inference:**

The primary scoring mechanism. Risk is calculated using the Sigmoid function: $P = 1 / (1 + \exp(-z))$, where z is the weighted sum of over 20+ signals plus a learned bias.

- **Error Level Analysis (ELA):**

A pixel-diff strategy where images are resaved at 90% quality and compared to the original. We calculate the `artifactCount / totalPixels` ratio to detect digital manipulation.

- **Fuzzy Keyword Matching:**

Adversarial defense using non-alphanumeric striping. It ensures "P-a-y-m-e-n-t" matches the "payment" rule by calculating matches on stripped string vectors.

- **GST Validator (Mod-36):**

A mathematical validation of Indian Business IDs using the official checksum algorithm. Each character is mapped to a code point and verified against a weighted sum modulo 36.

- **K-Fold Cross-Validation:**

Training stability is ensured by splitting the dataset into 5 unique "folds" and validating that precision remains consistent across all subsets.

2. Machine Learning Tech Stack

Python Layer (Training & Retraining)

- **Scikit-Learn:**

Used for training our Layer 1 Logistic Regression models, performing Stratified K-Fold validation, and calculating precision metrics.

- **Pandas & NumPy:**

The backbone for data manipulation. Used to clean MongoDB exports and perform high-speed vector calculations for model weights.

- **PyMongo:**

Connects the Python training scripts directly to our scan production database for automated model updates.

Node.js Layer (Production Inference)

- **Tesseract.js:**

Provides the OCR engine. We utilize a "Multi-Pass" strategy (Standard vs Thresholded) for maximum accuracy on low-quality scans.

- **Jim:**

Advanced image processing library used for normalization, contrast stretching, and pixel-level forgery analysis.

- **pdfjs-dist:**

High-performance PDF brain used to render document pages into high-resolution